

Because you can put variable names and expressions inline inside the string, your code becomes more readable than using the old ways of string formatting.

All of these different ways to format strings go against the Zen of Python aphorism that there should be one—and preferably only one—obvious way to do something. But f-strings are an improvement to the language (in my opinion), and as the other guideline states, practicality beats purity. If you're writing code for Python 3.6 or later only, use f-strings. If you're writing code that might be run by earlier Python versions, stick to the `format()` string method or `%s` conversion specifiers.

Making Shallow Copies of Lists

The *slice* syntax can easily create new strings or lists from existing ones. Enter the following into the interactive shell to see how it works:

```
>>> 'Hello, world!'[7:12] # Create a string from a larger string.
'world'
>>> 'Hello, world!':[5] # Create a string from a larger string.
'Hello'
>>> ['cat', 'dog', 'rat', 'eel'][2:] # Create a list from a larger list.
['rat', 'eel']
```

The colon (:) separates the starting and ending indexes of the items to put in the new list you're creating. If you omit the starting index before the colon, as in `'Hello, world!':[5]`, the starting index defaults to 0. If you omit the ending index after the colon, as in `['cat', 'dog', 'rat', 'eel'][2:]`, the ending index defaults to the end of the list.

If you omit both indexes, the starting index is 0 (the start of the list) and the ending index is the end of the list. This effectively creates a copy of the list:

```
>>> spam = ['cat', 'dog', 'rat', 'eel']
>>> eggs = spam[:]
>>> eggs
['cat', 'dog', 'rat', 'eel']
>>> id(spam) == id(eggs)
False
```
